

2024年度後期輪読会 #10 (2024/12/19)

YOLOX

京都大学工学部情報学科 数理工学コース B3

浦辺 晴基

- YOLOXの概要
- アーキテクチャ
- YOLOシリーズからの改善点
- 他手法との性能比較

- YOLOXの概要
- アーキテクチャ
- YOLOシリーズからの改善点
- 他手法との性能比較

YOLOXの概要

■ YOLOXとは

YOLOXは、2021年に論文[YOLOX: Exceeding YOLO Series in 2021](#)で提案された、

YOLOv3をベースラインにして改善を加えたリアルタイム物体検出モデルである。

実装は<https://github.com/Megvii-BaseDetection/YOLOX>を参照。

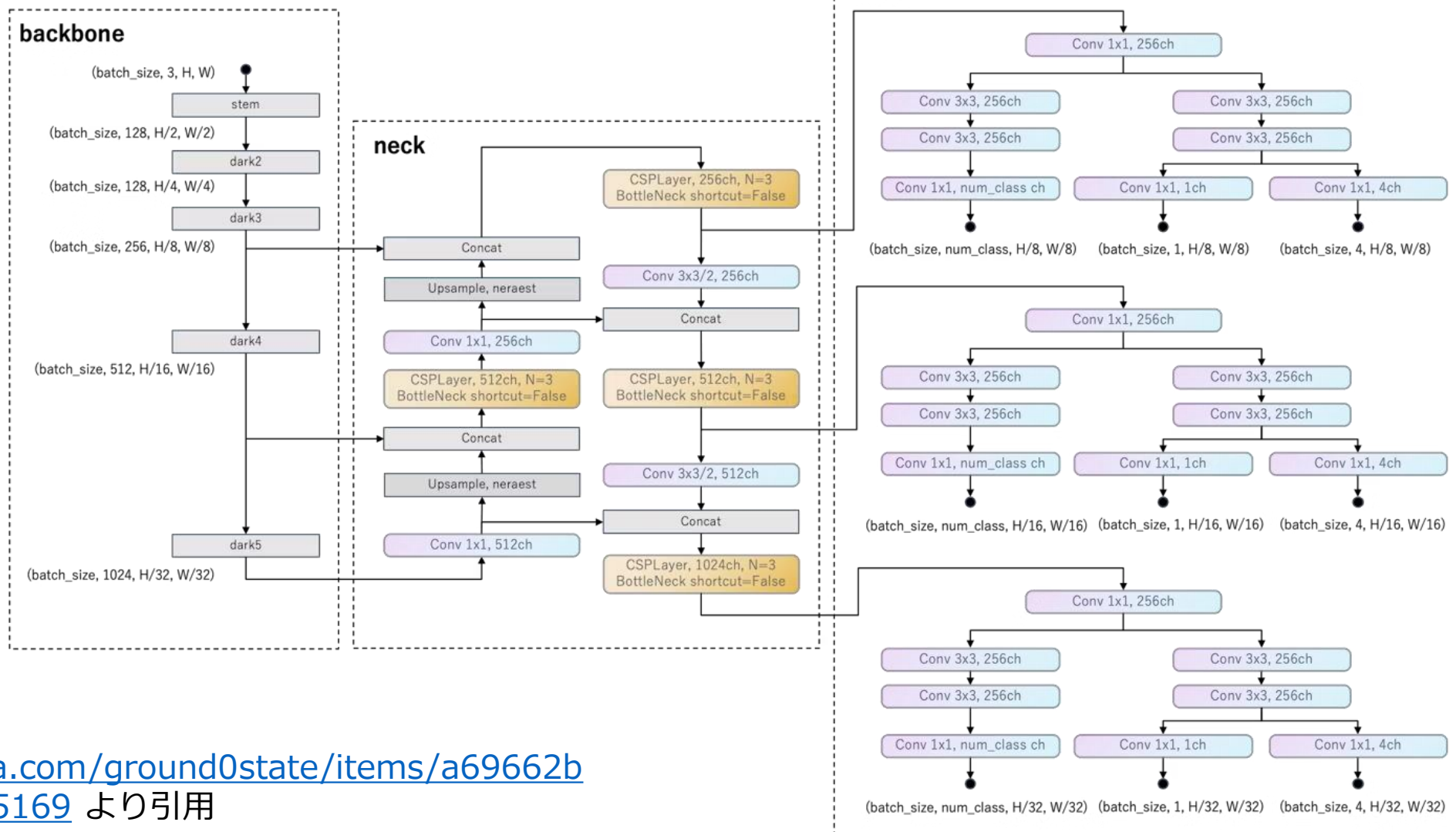
■ YOLOシリーズとYOLOXの相違点

YOLOシリーズ	YOLOX
アンカーベース検出	アンカーフリー検出
Coupled Head (分類と回帰を同じヘッドで行う)	Decoupled Head (分類と回帰を異なるヘッドで行う)
ラベル割り当てに柔軟性がない・非効率	SimOTAの導入
標準的なデータ拡張	Mosaic・MixUpの導入（強力なデータ拡張）

- YOLOXの概要
- **アーキテクチャ**
- YOLOシリーズからの改善点
- 他手法との性能比較

YOLOXのアーキテクチャ

■ ネットワーク全体

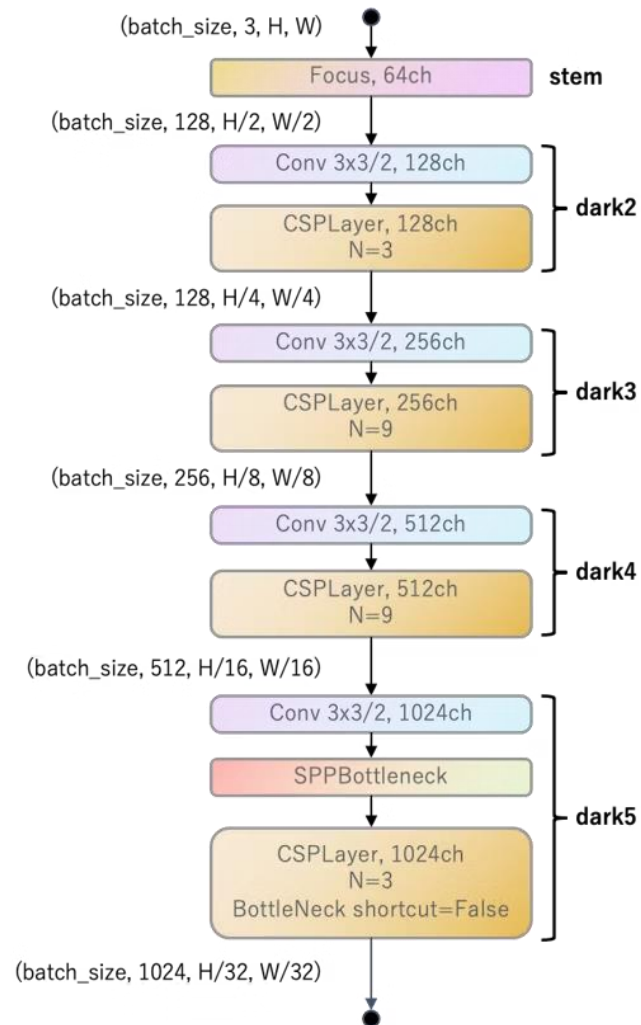


<https://qiita.com/ground0state/items/a69662bc6aa2eb715169> より引用

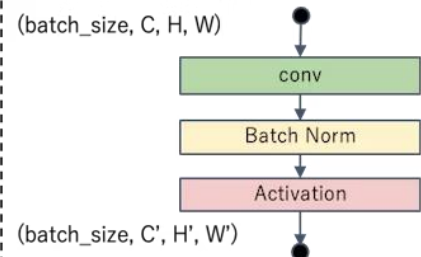
YOLOXのアーキテクチャ

■ Backbone

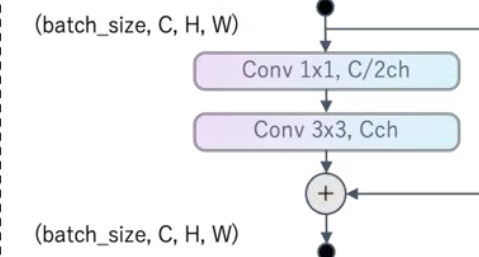
CSPDarknet53



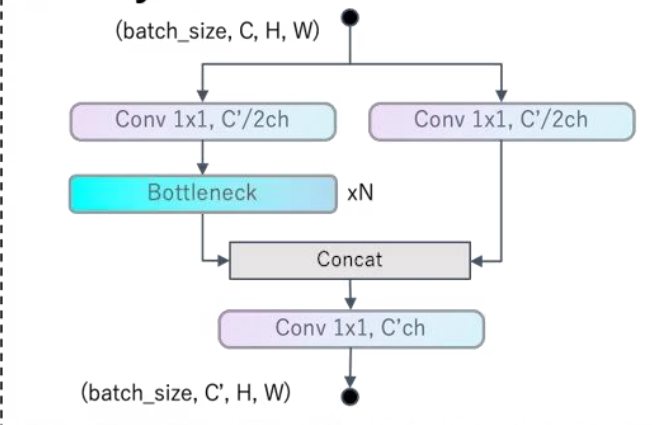
Conv



Bottleneck



CSPLayer



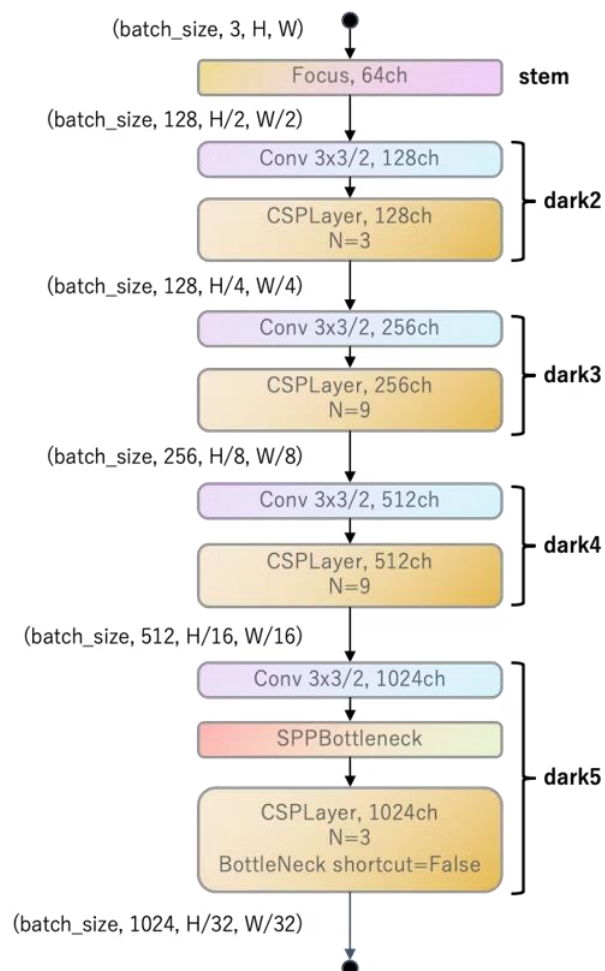
<https://qiita.com/ground0state/items/a69662bc6aa2eb715169> より引用

YOLOXのアーキテクチャ

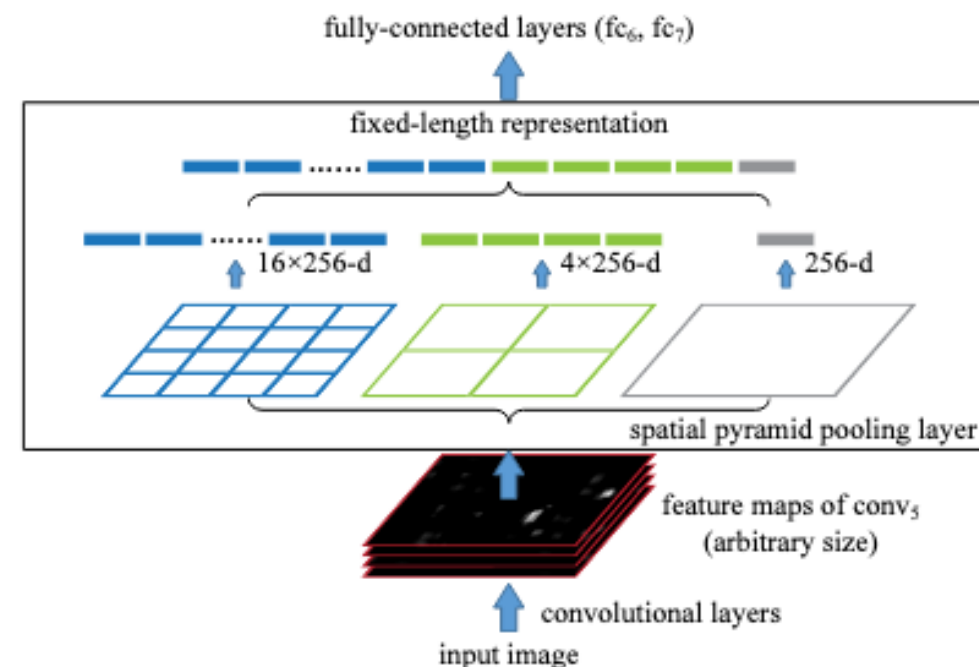
■ Backbone

• Darknet53

CSPDarknet53



- **SPP (Spatial Pyramid Pooling) 層**
入力画像のサイズに関わらず
特徴マップを固定サイズの出力に変換する

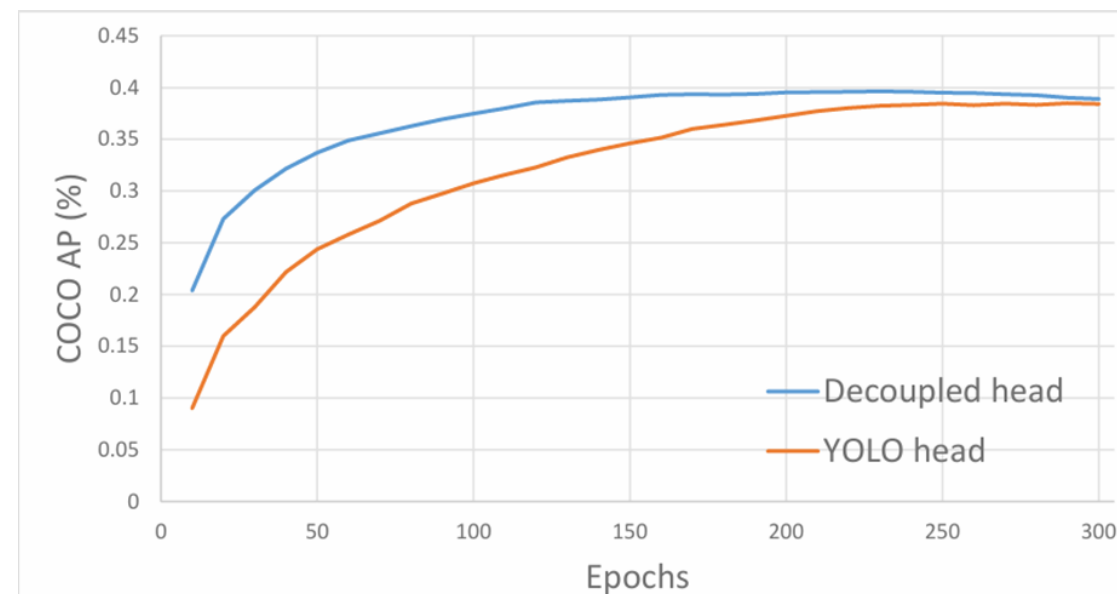
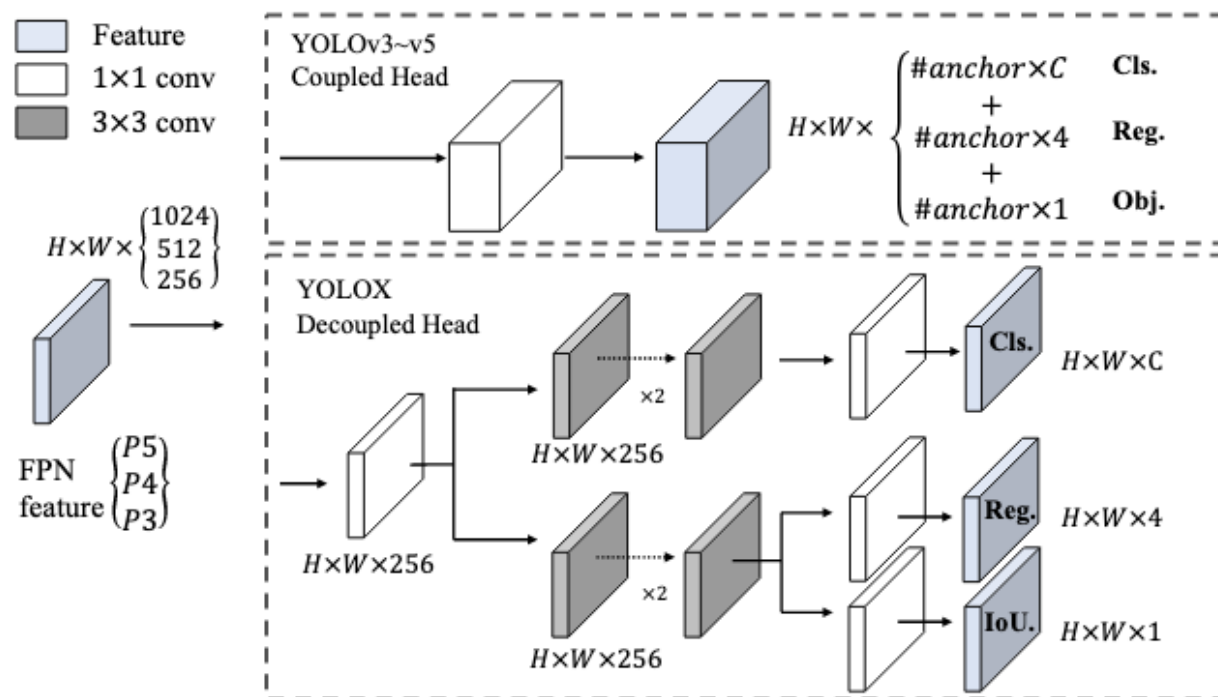


YOLOXのアーキテクチャ

Decoupled Head

クラス分類、バウンディングボックス回帰、IoUスコア予測をそれぞれ個別のHeadで行う。

従来のCoupled Headに比べて収束が早く、効率的に高い精度を出せる。



- YOLOXの概要
- アーキテクチャ
- YOLOシリーズからの改善点
- 他手法との性能比較

YOLOXの改善点

■ Strong data augmentation

RandomFlip（ランダムな方向に反転）やColorJitter（色調をランダムに変化）などに加え、**Mosaic**や**MixUp**といった強力なデータ拡張を適用している。

➤ Mosaic

右図のように、4つの画像を混ぜて
新たなデータセットを作成する。



aug_-319215602_0_-238783579.jpg



aug_-1271888501_0_-749611674.jpg



aug_1462167959_0_-1659206634.jpg



aug_1474493600_0_-45389312.jpg



aug_1715045541_0_603913529.jpg



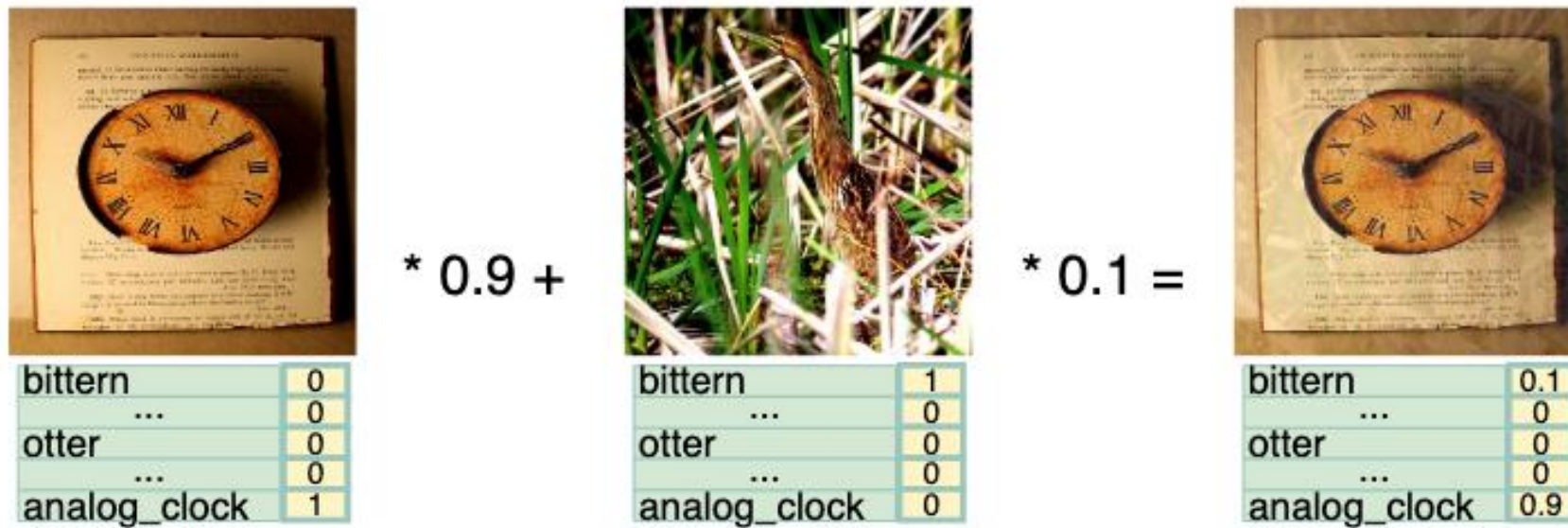
aug_1779424844_0_-589696888.jpg

YOLOXの改善点

■ Strong data augmentation

➤ MixUp

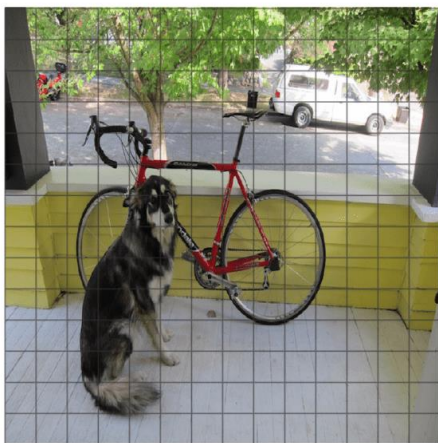
下図のように、2つの画像とラベルをある割合で合成して新たなデータセットを作成する。



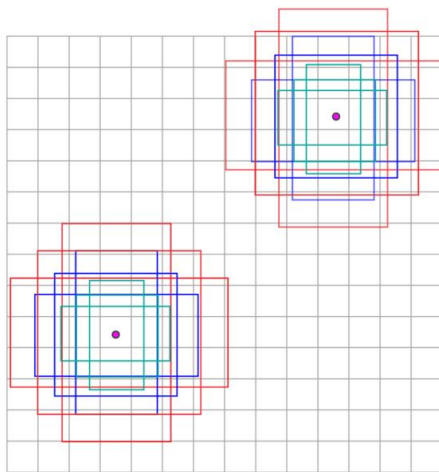
YOLOXの改善点

■ Anchor-free

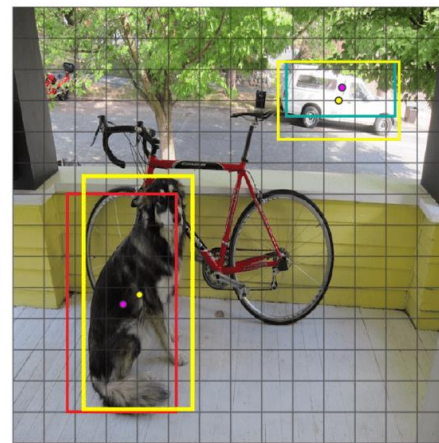
従来のYOLOシリーズでは、**アンカーベース**（中心と縦横比があらかじめ定義されたアンカーボックスという矩形に基づいてバウンディングボックスの予測を行う手法）を用いていたが、計算コストの増加や、適切なサイズのアンカーボックスが存在しない場合などに課題があった。



448 x 448 の入力画像と対応する
特徴マップ ([13 x 13]) のセル配列。
※アンカー・＝各セルの中心点



特徴マップ上の各アンカー・に
配置されたアンカーボックス群
(3 スケール × 3 アスペクト比)



アンカーボックスからの回帰に
よる予測ボックス (黄色)

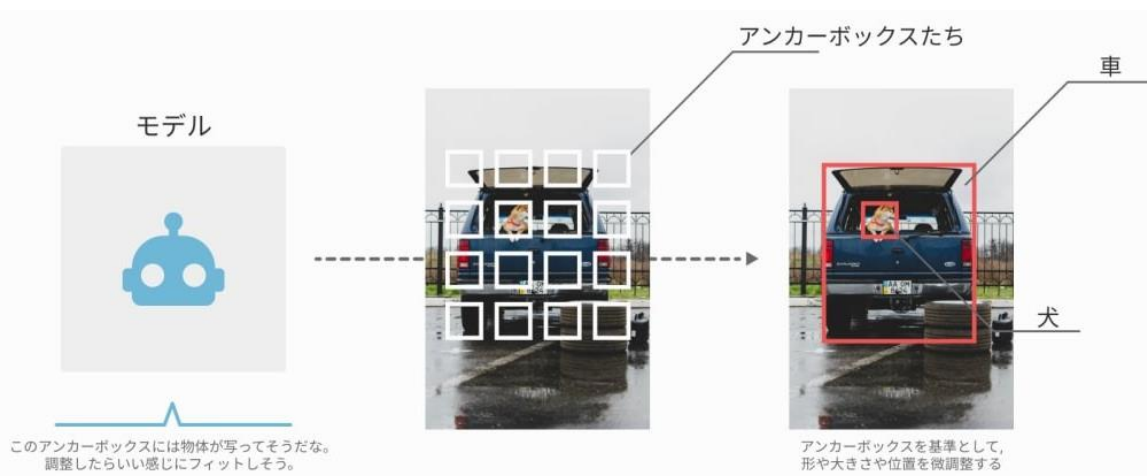
<https://cvml-expertguide.net/terms/dl/object-detection/anchor-box/>より引用

YOLOXの改善点

■ Anchor-free

一方、YOLOXでは**アンカーフリー**を採用しており、アンカーボックスを用意せずに特徴マップから直接バウンディングボックスの位置や幅、高さを予測する。

アンカーベース



アンカーフリー



<https://zenn.dev/nekoallergy/articles/dl-advanced-anchor>より引用

YOLOXの改善点

■ Multi positives

従来のラベル割り当てでは、

「正解データ（GT）のバウンディングボックスの中心があるアンカーのみをPositiveとして評価する」
という手法を採用していたが、PositiveとNegativeの数が著しく不均衡になってしまう問題があった。

そこでYOLOXにおけるラベル割り当てでは、

「正解データ（GT）のバウンディングボックスの中心があるアンカーの
近傍3×3アンカーもPositiveとして評価する」

という手法を採用することで、Positiveの数を9倍にし、データ不均衡の問題に対処している。

YOLOXの改善点

■ SimOTA

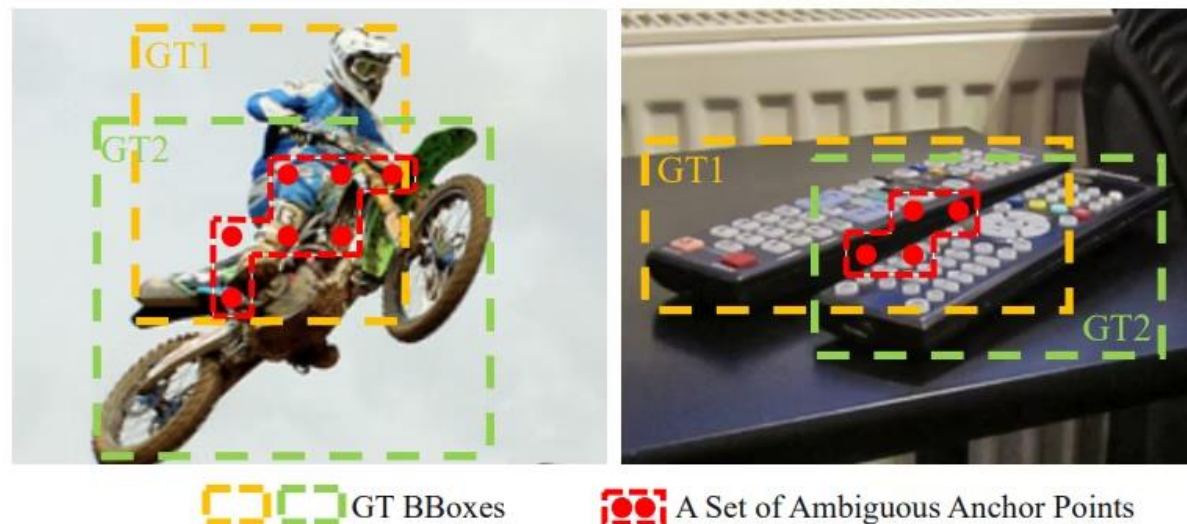
右下図のように複数の正解ボックスが重なり合っている場合には、予測したバウンディングボックスに対して、どれを割り当てるべきかが曖昧なアンカーが存在する。IoUの閾値で割り当てる従来の手法は柔軟性に欠け、問題があった。

そこで、この正解ボックスと予測ボックスの割り当てを**最適輸送問題**として捉え、

Sinkhorn-Knoppアルゴリズムによって解く

OTA (Optimal Transport Assignment)

という手法が提案された。



YOLOXの改善点

■ SimOTA

➤ 最適輸送問題

m 人の供給者と n 人の需要者がいると仮定する。

供給者 i は s_i 個の荷物を保有し、

需要者 j は d_j 個の荷物を必要としているとする。

また、供給者 i から需要者 j への

各荷物の輸送コストを c_{ij} とする。

最小の輸送コストで供給者からのすべての荷物を需要者へと輸送できる

最適輸送計画 $\pi^* = \{\pi_{ij} \mid i = 1, 2, \dots, m, j = 1, 2, \dots, n\}$ を求める線形計画問題。

$$\begin{aligned} \min_{\pi} \quad & \sum_{i=1}^m \sum_{j=1}^n c_{ij} \pi_{ij} \\ \text{s.t.} \quad & \sum_{i=1}^m \pi_{ij} = d_j, \quad \sum_{j=1}^n \pi_{ij} = s_i, \\ & \sum_{i=1}^m s_i = \sum_{j=1}^n d_j, \\ & \pi_{ij} \geq 0, \quad i = 1, 2, \dots, m, j = 1, 2, \dots, n \end{aligned}$$

■ SimOTA

OTAでは正解ボックスと予測ボックスの割り当てを最適輸送問題として捉え、

供給者 : k 個のPositiveラベルを保持する正解ボックス & $n - m \times k$ 個のNegativeラベルを保持する背景

需要者 : 1個のPositiveラベルを必要とする予測ボックス

と考えるので、右のように決まる。

$$s_i = \begin{cases} k & (i = 1, 2, \dots, m) \\ n - m \times k & (i = m + 1) \end{cases}$$

$$d_j = 1 \quad (j = 1, 2, \dots, n)$$

また、正解ボックス G_i から予測ボックス P_j への輸送コストは以下のように定義される。

$$\begin{cases} c_{ij}^{fg} = L^{cls}(P_j^{cls}(\theta), G_i^{cls}) + \lambda L^{reg}(P_j^{box}(\theta), G_i^{box}) \\ c_{ij}^{bg} = L^{cls}(P_j^{cls}(\theta), \emptyset) \end{cases}$$

YOLOXの改善点

■ SimOTA

➤ Sinkhorn-Knoppアルゴリズム

最適輸送問題は線形計画問題のため、単体法や内点法により厳密解を得ることは可能であるが、計算コストの問題がある。そのためOTAでは、近似解を高速に得るためのアルゴリズムとして

Sinkhorn-Knoppアルゴリズムを採用している。

本アルゴリズムでは右のように

最適輸送問題の目的関数に

エントロピー正則化項を加えた凸関数を

目的関数とする緩和問題を考える。

$$\begin{aligned} \min_{\pi} \quad & \sum_{i=1}^m \sum_{j=1}^n c_{ij} \pi_{ij} + \gamma \{ \pi_{ij} (\log \pi_{ij} - 1) \} \\ \text{s.t.} \quad & \sum_{i=1}^m \pi_{ij} = d_j, \quad \sum_{j=1}^n \pi_{ij} = s_i, \\ & \sum_{i=1}^m s_i = \sum_{j=1}^n d_j, \\ & \pi_{ij} \geq 0, \quad i = 1, 2, \dots, m, \quad j = 1, 2, \dots, n \end{aligned}$$

YOLOXの改善点

■ SimOTA

➤ Sinkhorn-Knoppアルゴリズム

ラグランジュの未定乗数法により、

緩和問題の目的関数は以下の非制約目標に変換される：

$$\min_{\pi} \sum_{i=1}^m \sum_{j=1}^n c_{ij} \pi_{ij} + \gamma \{ \pi_{ij} (\log \pi_{ij} - 1) \} \\ + \alpha_j \left(\sum_{i=1}^m \pi_{ij} - d_j \right) + \beta_i \left(\sum_{j=1}^n \pi_{ij} - s_i \right)$$

最適輸送計画は以下のように得られる：

$$\pi_{ij}^* = \exp \left(-\frac{\alpha_j}{\gamma} \right) \exp \left(-\frac{c_{ij}}{\gamma} \right) \exp \left(-\frac{\beta_i}{\gamma} \right)$$

Algorithm 1 Sinkhorn-Knopp Algorithm

Input:

s is the supplying vector

d is the demanding vector

c is the cost matrix

$\lambda > 0$ is the regularization hyperparameter

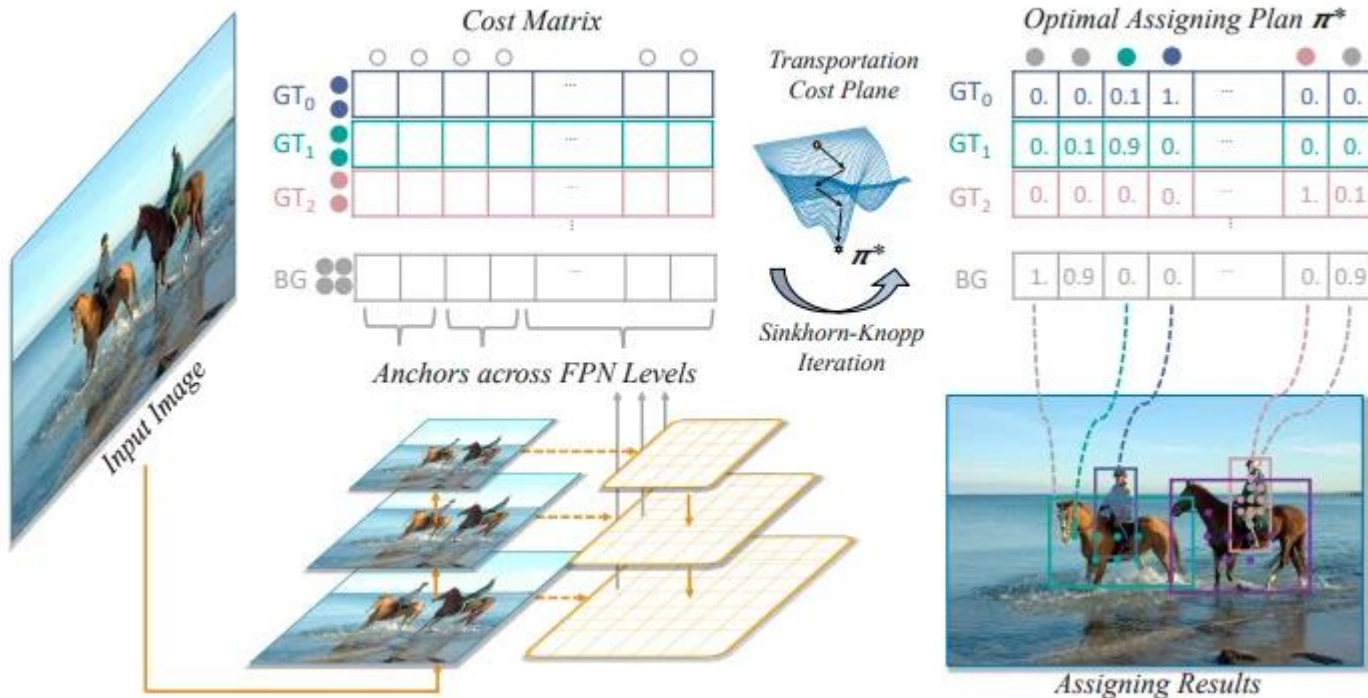
Output:

π^* is the assigning plan

- 1: Compute $M \in \mathbb{R}^{m \times n}$ where $M_{ij} \leftarrow \exp(-c_{ij}/\gamma)$
 - 2: Initialize $u \in \mathbb{R}^m$ as $u \leftarrow [1, 1, \dots, 1]$
 - 3: Initialize $v \in \mathbb{R}^n$ as $v \leftarrow [1, 1, \dots, 1]$
 - 4: **while** not convergence **do**
 - 5: $u \leftarrow s./(Mv)$ ▷ Element-wise division
 - 6: $v \leftarrow d./(M^\top u)$ ▷ Element-wise division
 - 7: **end while**
 - 8: $\pi^* \leftarrow \text{diag}(v)M\text{diag}(u)$
-

YOLOXの改善点

■ SimOTA



Algorithm 1 Optimal Transport Assignment (OTA)

Input:

- I is an input image
- A is a set of anchors
- G is the *gt* annotations for objects in image I
- γ is the regularization intensity in Sinkhorn-Knopp Iter.
- T is the number of iterations in Sinkhorn-Knopp Iter.
- α is the balanced coefficient in Eq. 2

Output:

π^* is the optimal assigning plan

- 1: $m \leftarrow |G|, n \leftarrow |A|$
- 2: $P^{\text{cls}}, P^{\text{box}} \leftarrow \text{Forward}(I, A)$
- 3: $s_i (i = 1, 2, \dots, m) \leftarrow \text{Dynamic } k \text{ Estimation}$
- 4: $s_{m+1} \leftarrow n - \sum_{i=1}^m s_i$
- 5: $d_j (j = 1, 2, \dots, n) \leftarrow \text{OnesInit}$
- 6: pairwise *cls* cost: $c_{\text{cls}}^{ij} = \text{FocalLoss}(P_j^{\text{cls}}, G_i^{\text{cls}})$
- 7: pairwise *reg* cost: $c_{\text{reg}}^{ij} = \text{IoULoss}(P_j^{\text{box}}, G_i^{\text{box}})$
- 8: pairwise Center Prior cost: $c_{ij}^{\text{cp}} \leftarrow (A_j, G_i^{\text{box}})$
- 9: *bg cls* cost: $c_{\text{cls}}^{\text{bg}} = \text{FocalLoss}(P_j^{\text{cls}}, \emptyset)$
- 10: *fg* cost: $c^{\text{fg}} = c_{\text{cls}} + \alpha c_{\text{reg}} + c_{\text{cp}}$
- 11: compute final cost matrix c via concatenating $c_{\text{cls}}^{\text{bg}}$ to the last row of c^{fg}
- 12: $v^0, u^0 \leftarrow \text{OnesInit}$
- 13: **for** $t=0$ **to** T **do**:
- 14: $u^{t+1}, v^{t+1} \leftarrow \text{SinkhornIter}(c, u^t, v^t, s, d)$
- 15: compute optimal assigning plan π^* according to Eq. 11
- 16: **return** π^*

YOLOXの改善点

■ SimOTA

OTAは高性能な割り当て戦略だが、計算量や学習時間の増大につながるため、

YOLOXではOTAを簡略化した**SimOTA** (Simple Optimal Transport Assignment) を採用している。

SimOTAは正解ボックスが保持するPositiveラベルの数 k の適切な値を動的に決定する**Dynamic K戦略**が特徴であり、正解ボックス G_i と各予測ボックスのIoUに基づいた以下の式で k が動的に決定される。

$$k = \max \left(\sum_{j \in \text{Top-10}} \text{IoU}_{ij}, 1 \right)$$

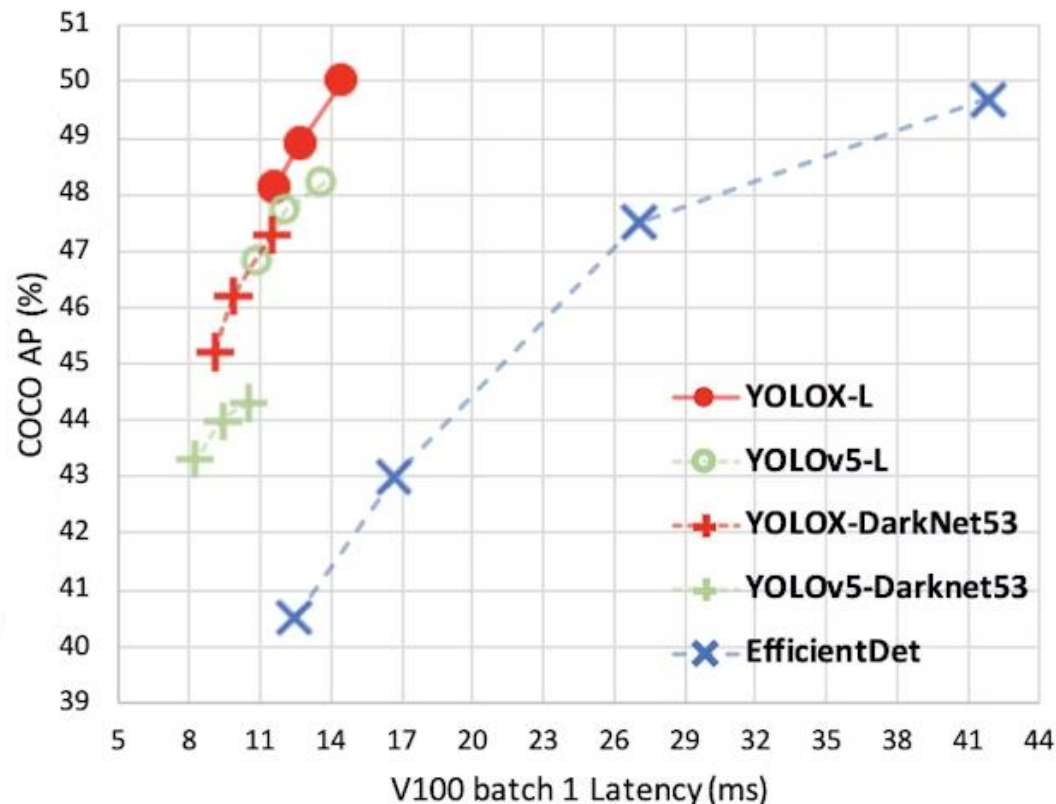
予測ボックスに複数の正解ボックスが割り当てられた場合は、

複雑な計算をせずにコストが最小の正解ボックスを割り当てて修正する。

Sinkhorn-Knoppアルゴリズムを用いていないため、効率的になっている。

- YOLOXの概要
- アーキテクチャ
- YOLOシリーズからの改善点
- 他手法との性能比較

精度と推論速度のトレードオフ



精度とモデルサイズのトレードオフ

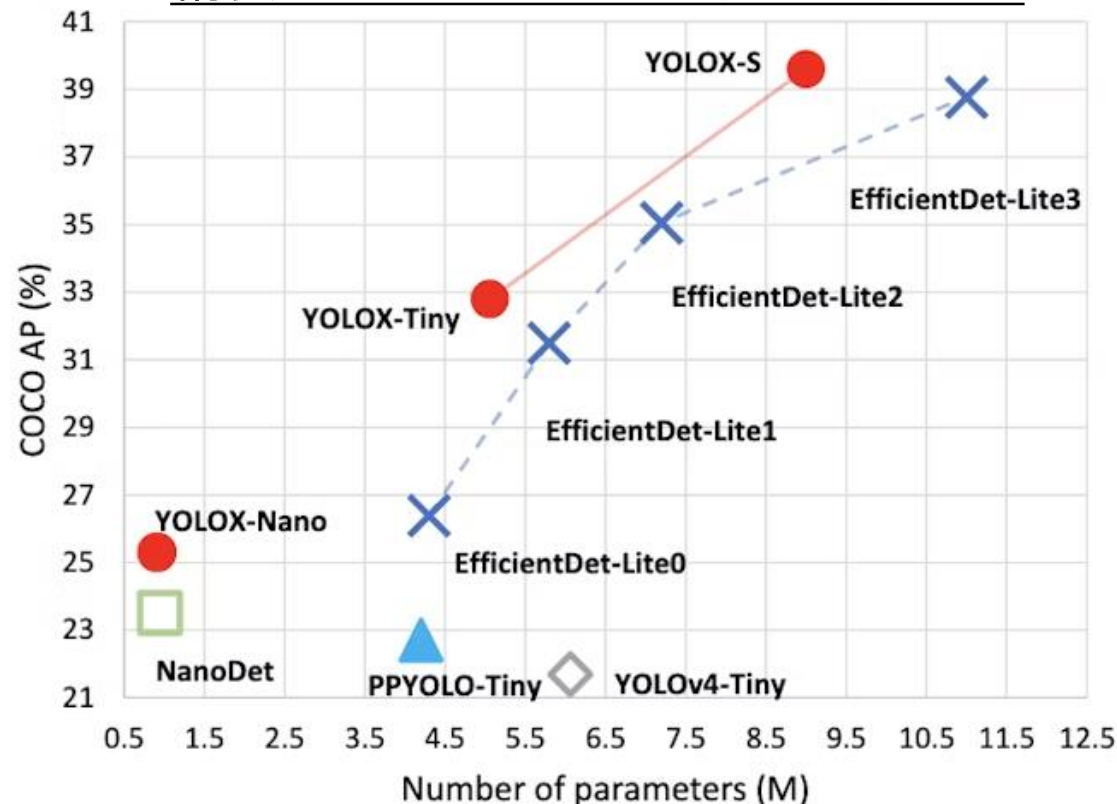


Figure 1: Speed-accuracy trade-off of accurate models (top) and Size-accuracy curve of lite models on mobile devices (bottom) for YOLOX and other state-of-the-art object detectors.

Method	Backbone	Size	FPS (v100)	AP (%)	IoUの閾値		物体のサイズ		
					AP ₅₀	AP ₇₅	AP _S	AP _M	AP _L
YOLOv3 + ASFF* [18]	Darknet-53	608	45.5	42.4	63.0	47.4	25.5	45.7	52.3
YOLOv3 + ASFF* [18]	Darknet-53	800	29.4	43.9	64.1	49.2	27.0	46.6	53.4
EfficientDet-D0 [28]	Efficient-B0	512	98.0	33.8	52.2	35.8	12.0	38.3	51.2
EfficientDet-D1 [28]	Efficient-B1	640	74.1	39.6	58.6	42.3	17.9	44.3	56.0
EfficientDet-D2 [28]	Efficient-B2	768	56.5	43.0	62.3	46.2	22.5	47.0	58.4
EfficientDet-D3 [28]	Efficient-B3	896	34.5	45.8	65.0	49.3	26.6	49.4	59.8
PP-YOLOv2 [11]	ResNet50-vd-dcn	640	68.9	49.5	68.2	54.4	30.7	52.9	61.2
PP-YOLOv2 [11]	ResNet101-vd-dcn	640	50.3	50.3	69.0	55.3	31.6	53.9	62.4
YOLOv4 [1]	CSPDarknet-53	608	62.0	43.5	65.7	47.3	26.7	46.7	53.3
YOLOv4-CSP [30]	Modified CSP	640	73.0	47.5	66.2	51.7	28.2	51.2	59.8
YOLOv3-ultralytics ²	Darknet-53	640	95.2	44.3	64.6	-	-	-	-
YOLOv5-M [7]	Modified CSP v5	640	90.1	44.5	63.1	-	-	-	-
YOLOv5-L [7]	Modified CSP v5	640	73.0	48.2	66.9	-	-	-	-
YOLOv5-X [7]	Modified CSP v5	640	62.5	50.4	68.8	-	-	-	-
YOLOX-DarkNet53	Darknet-53	640	90.1	47.4	67.3	52.1	27.5	51.5	60.9
YOLOX-M	Modified CSP v5	640	81.3	46.4	65.4	50.6	26.3	51.0	59.9
YOLOX-L	Modified CSP v5	640	69.0	50.0	68.5	54.5	29.8	54.5	64.4
YOLOX-X	Modified CSP v5	640	57.8	51.2	69.6	55.7	31.2	56.1	66.1

Table 6: Comparison of the speed and accuracy of different object detectors on COCO 2017 *test-dev*. We select all the models trained on 300 epochs for fair comparison.

まとめ

物体検出モデルの一種である**YOLOX**について解説した。

YOLOX とは

- YOLOv3をベースラインにして改善を加えたリアルタイム物体検出モデル
- 精度と推論速度・精度とモデルサイズのトレードオフにおいて優れている

アーキ テクチャ

- BackboneにDarkNet53とSPP層を採用
- 分類、回帰、スコア予測をそれぞれ個別のHeadで行う**Decoupled Head**

改善点

- 強力なデータ拡張(**Mosaic & MixUp**)
- アンカーフリー
- **Multi positives**と**SimOTA**によるラベル割り当て

1. YOLOX: Exceeding YOLO Series in 2021. (<https://arxiv.org/abs/2107.08430>)
2. YOLOv3: An Incremental Improvement. (<https://arxiv.org/abs/1804.02767>)
3. YOLOv4: Optimal Speed and Accuracy of Object Detection. (<https://arxiv.org/abs/2004.10934>)
4. Spatial Pyramid Pooling in Deep Convolutional Networks for Visual Recognition.
(<https://arxiv.org/abs/1406.4729>)
5. Bag of Freebies for Training Object Detection Neural Networks.
(<https://arxiv.org/abs/1902.04103>)
6. OTA: Optimal Transport Assignment for Object Detection. (<https://arxiv.org/abs/2103.14259>)
7. YOLOXのアーキテクチャ. Qiita. (<https://qiita.com/ground0state/items/a69662bc6aa2eb715169>)
8. 実装から見るYOLOX : 2021年のYOLOシリーズを超えて. Qiita.
(<https://qiita.com/koshian2/items/af032cb102f48e789e66>)

参考文献

9. 【論文5分まとめ】 YOLOx: Exceeding yolo series in 2021. Zenn.
(<https://zenn.dev/takoroy/articles/e1e401b865f6c1>)
10. 【論文5分まとめ】 Ota: Optimal transport assignment for object detection. Zenn.
(<https://zenn.dev/takoroy/articles/dd511b8f8710f3>)
11. 最新のリアルタイム物体検出モデルYOLOXの紹介. 株式会社アイビス.
(<https://www.ibis.ne.jp/blog/1992-2/>)
12. 【CVPR2021】 物体検出のラベルアサインメントを最適化するOTAを紹介！. DeepSquare Media.
(<https://deepsquare.jp/2021/08/ota-object-detection/>)
13. 【物体検出】 アンカーボックスって？（アンカーベースとアンカーフリー） 【図解】 . Zenn.
(<https://zenn.dev/nekoallergy/articles/dl-advanced-anchor>)
14. アンカーボックス (Anchor Box) [物体検出]. CVMLエキスパートガイド. (<https://cvml-expertguide.net/terms/dl/object-detection/anchor-box/>)